How Spring Boot Builds Your Application Using Configuration ??

(application.properties vs application.yml + pom.xml + starters + auto-configuration)

Why Configuration Exists ?

In enterprise systems, code is not the only thing that changes.

Environment, ports, databases, credentials, logging levels, feature flags change much more frequently than Java code.

Spring Boot separates:

- **What the app does** → Java code
- **How the app runs** → Configuration

That is why configuration is externalized into:

- pom.xml
- application.properties
- application.yml
- environment variables
- config folders

Spring Boot then converts all of this into a **runtime Environment** that controls:

- which beans exist
- which server runs
- which database connects
- which auto-configs activate

What is application.properties?

application.properties is the **control panel of your application**.

It lives in:

src/main/resources/application.properties

It is loaded **during Spring Boot startup** — before any beans are created.

Example:

```properties
server.port=9096
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.jpa.show-sql=true
logging.level.root=INFO
```

This overrides Spring Boot defaults:

Default	Your override
8080	9096
H2	MySQL
minimal logs	detailed logs

Spring Boot reads this file and builds a giant map called **Environment**.

Custom Properties

You can define your own keys:

```
myapp.name=Order Service
myapp.owner=TVS
```

Read them:

```
@Value("${myapp.name}")
private String appName;
```

Why we added @Value annotation?

→ So values change **without recompiling Java**.

Why @Value Exists

Without @Value, Java code is **hard-wired**.

With @Value, configuration becomes **externalized**.

This allows:

- different DBs in prod vs dev
- different ports
- different credentials
- different feature toggles

Custom Properties File

Spring Boot only auto-loads:

```
application.properties
application.yml
```

Not:

```
myfile.properties
db.properties
```

To load custom properties files:

Use @PropertySource Annotation with file name/path

```
@PropertySource("classpath:myfile.properties")
```

If file is in:

```
src/main/resources/myfolder/myfile.properties
```

Use:

```
@PropertySource("classpath:myfolder/myfile.properties")
```

Add below @SpringBootApplication in Starter app → Spring adds it into **Environment**.

Where Spring Boot Searches for application.properties ?

Spring Boot searches **in this order**:

```
1) ./config/application.properties
2) ./application.properties
3) classpath:/config/application.properties
4) classpath:/application.properties
```

So:

```
src/main/resources/config/application.properties
```

is valid
But:

```
src/main/resources/abc/application.properties
```

is ignored unless you use @PropertySource.

External Config Folder

You can place:

```
config/application.properties
```

next to your JAR.

Spring Boot auto-detects it.

This is how DevOps configures prod without touching code.

Why file must be named application.properties ?

Spring Boot auto-loader looks for **only one name**:

application

Anything else is ignored unless explicitly imported.

YAML

YAML = structured configuration.

Example:

```yaml
server:
  port: 9096

spring:
  datasource:
    url: jdbc:mysql://localhost:3306/mydb
```

Same as:

```
server.port=9096
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
```

YAML is preferred for complex configs.

properties vs yaml

Properties	**YAML**
Flat	Hierarchical
Hard to read	Clean
Good for small	Best for large

Startup & Configuration Flow

From Spring Initializr:

You get:

```
pom.xml
src/main/resources/application.properties
```

Why?
Because **Spring Boot needs configuration before code runs**.

Startup:

```
main() → SpringApplication.run()
→ Read pom dependencies
→ Load application.properties
→ Build Environment
→ Trigger auto-configurations
→ Create beans
→ Start Tomcat
→ App runs
```

Loaded **once per startup**.

How pom.xml, starters & properties work together ?

pom.xml:

```
spring-boot-starter-web
spring-boot-starter-data-jpa
mysql-connector-j
```

Spring Boot sees:

```
Web → start Tomcat
JPA → start Hibernate
MySQL → configure datasource
```

application.properties controls:

```
URL
username
password
dialect
port
logging
```

Environment → Auto-configuration → Beans → Server

Good vs Broken Example

Working:

```
spring.datasource.url=jdbc:mysql://localhost/db
```

Broken:

```
spring.datasource.urll=jdbc:mysql://localhost/db
```

Auto-config fails because key is wrong.

Hidden Traps

- Misspelled properties silently fail
- Multiple config files override each other
- YAML indentation errors break parsing
- External configs override classpath
- Profiles (application-dev.properties) change everything

Interview Perspective

Interviewers test:

- Do you know **who controls Spring Boot**
- Do you understand **why config drives beans**
- Can you debug startup failures

Interview Questions

1. When is application.properties loaded?

   → During bootstrap before beans.

2. Why must it be named application.properties?

   → Spring Boot auto-loader.

3. Where is it searched?

   → classpath, config, external folder.

4. How does DB get configured?

   → pom + properties → auto-config.

5. What is Environment?

   → Key-value map of all configuration.

6. YAML vs properties?

   → Same values, different format.

7. What if property is missing?

   → Default used or bean fails.

8. Can it live outside jar?

   → Yes, in config folder.

9. What does starter do?

   → Pulls correct dependency set.

10. What drives auto-configuration?

    → Environment + classpath.

One-Page Mental Model

```
pom.xml → what tech exists
application.properties → how they behave
Environment → merged config
Auto-config → creates beans
ApplicationContext → runs app
```

Enterprise Relevance

This is how:

- prod differs from dev
- cloud overrides ports
- Kubernetes injects secrets
- CI/CD deploys without rebuilds

Final Summary — Story

You generate a Spring Boot project.
→ It gives you pom.xml and application.properties.

You add:

```
spring-boot-starter-web
spring-boot-starter-data-jpa
mysql
```

You write:

```
spring.datasource.url
server.port
```

When the app starts:

Spring Boot:
→ Reads pom
→ Loads application.properties
→ Builds Environment
→ Activates auto-config
→ Creates Tomcat, DB, JPA, Controllers
→ Runs your app

You never wired anything manually — configuration drove everything.

Definitions

Term	Meaning
pom.xml	Declares what technologies your app uses
Starter	A curated bundle of dependencies
application.properties	Flat key-value configuration
application.yml	Structured configuration format
Environment	Spring's in-memory map of all config values
Auto-configuration	Spring Boot creating beans based on environment
Bean	A Java object managed by Spring
Container (ApplicationContext)	Holds and wires all beans

XML → *eXtensible Markup Language*
YAML → *YAML Ain't Markup Language*